

```
$ javac NotificationLab.java && java NotificationLab
```

# Java Interface

// 從行為契約到可替換依賴

**contract**

編譯期保證

**dispatch**

執行期選擇

**injection**

組裝端決策

```
$ cat learning-path.md
```

# 今日路線

## 01 / 契約

interface 與 implements

## 02 / 派送

介面型別與真實物件

## 03 / 依賴

constructor injection、DIP、ISP

## 04 / 演進

default、lambda、sealed

**目標：**能畫出依賴方向，也能用測試替身驗證高層流程。

PART 01

# 先找到耦合點

高層流程為什麼一直被低層細節拖著改？

# 第一版：能跑，但知道太多

```
1  class NotificationService {
2      void notify(String channel, String message) {
3          if (channel.equals("email")) {
4              new EmailClient().deliver(message);
5          } else if (channel.equals("sms")) {
6              new SmsClient().deliver(message);
7          }
8      }
9  }
```

## COUPLING REPORT

- × 知道有哪些管道
- × 知道 SDK 如何建立
- × 知道每個 SDK 的方法名

新增 Push → 修改高層流程

# 把共同需求命名



不同技術細節



DOMAIN LANGUAGE

**Notifier**

`send(message)`

介面先回答：呼叫端需要什麼行為？

# 宣告契約

```
1 public interface Notifier {  
2     void send(String message);  
3 }
```

## public abstract

一般無本體方法隱含

## no instance state

不能放每個物件的可變欄位

## role name

描述能力，不綁供應商

**contract** ≠ 實作；它是編譯器可檢查的承諾。

# implements : 履行承諾

```
1  final class EmailNotifier implements Notifier {
2      @Override
3      public void send(String message) {
4          System.out.println("EMAIL > " + message);
5      }
6  }
7
8  final class SmsNotifier implements Notifier {
9      @Override
10     public void send(String message) {
11         System.out.println("SMS > " + message);
12     }
13 }
```

實作方法必須是 `public` ; 不能把契約承諾縮成 `package-private`。

PART 02

# 同一型別，不同行為

編譯期看契約；執行期看物件

# 動態方法派送

```
1 Notifier notifier = new EmailNotifier();  
2 notifier.send("Welcome");  
3  
4 notifier = new SmsNotifier();  
5 notifier.send("Code: 8042");
```

COMPILE TIME

Notifier 有沒有 send ?

RUN TIME

真實物件是哪個實作？

同一個呼叫語法 → 不同的方法本體

# 從具體型別移到契約

```
1 final class NotificationService {
2     private final Notifier notifier;
3
4     NotificationService(Notifier notifier) {
5         this.notifier = notifier;
6     }
7
8     void notify(String message) {
9         notifier.send(message);
10    }
11 }
```

變化不只在型別：**建立哪個實作**的決策已移出服務。

# 建構子注入

```
1  final class NotificationService {  
2      private final Notifier notifier;  
3  
4      NotificationService(Notifier notifier) {  
5          this.notifier = notifier;  
6      }  
7  }
```

必要依賴明確、可保持 `final`，物件建好後就是有效狀態。

## 組裝端

```
1  public static void main(String[] args) {  
2      Notifier notifier =  
3          new EmailNotifier();  
4  
5      var service =  
6          new NotificationService(notifier);  
7  }
```

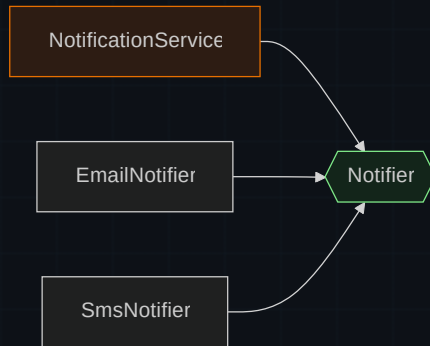
COMPOSITION ROOT

具體選擇集中在程式進入點。

# 依賴方向



直接依賴：高層指向低層細節



反轉後：兩側都指向契約

# 動手切換設計

拖曳我：新增管道時，哪個方塊必須改？

直接耦合

依賴契約

Email

SMS

Push

```
$ deps --graph
```

```
Service → Notifier ← implementations
```

新增管道 → 實作 Notifier，於組裝端接線

▶ dispatch()

```
// select channels, then dispatch
```

PART 03

# 原則不是口號

DIP、ISP 與可測試邊界

# 「用了 interface」≠「符合 DIP」

## 過度宣稱

每個具體類別加一個同名介面，所以架構已解耦。

## 較準確

由高層政策需要定義抽象，低層細節反過來實作它。

問的不是「有沒有關鍵字」，而是：抽象由誰主導？原始碼箭頭指向哪裡？

## CHECKPOINT\_01

哪一句最準確描述 DIP ?

A 只要宣告 `interface` , 就自動符合 DIP

B 高層政策與低層細節都依賴由高層需求主導的抽象

C 每個 `class` 都必須建立同名 `interface`

# ISP : 別逼實作者說謊

```
1 interface NotificationPlatform {  
2     void send(String message);  
3     void schedule(String message, Instant at);  
4     void revoke(String id);  
5 }
```

SMS 不支援撤回，卻被迫實作 `revoke` : 丟 `UnsupportedOperationException` 只是把設計問題延後到執行期。

Notifier

Schedulable

Revocable

# 組合：一個契約，多個委派

```
1  final class CompositeNotifier implements Notifier {
2      private final List<Notifier> delegates;
3
4      CompositeNotifier(List<Notifier> delegates) {
5          this.delegates = List.copyOf(delegates);
6      }
7
8      public void send(String message) {
9          for (Notifier notifier : delegates) {
10             notifier.send(message);
11         }
12     }
13 }
```

呼叫端仍只看到一個 `Notifier`。但要另外決定：某管道失敗時，停止還是繼續？

# 測試替身

```
1  final class RecordingNotifier
2      implements Notifier {
3
4      final List<String> messages =
5          new ArrayList<>();
6
7      public void send(String message) {
8          messages.add(message);
9      }
10 }
```

## 不需要真的寄信

```
1  @Test
2  void sendsMessage() {
3      var fake = new RecordingNotifier();
4      var service =
5          new NotificationService(fake);
6
7      service.notify("shipped");
8
9      assertEquals(
10         List.of("shipped"),
11         fake.messages);
12 }
```

fast · deterministic · offline

# Interface 還是 abstract class ?

| 問題    | Interface                | Abstract class |
|-------|--------------------------|----------------|
| 主要意圖  | 能力、邊界、替換                 | 共同狀態與骨架        |
| 實例欄位  | 不可                       | 可以             |
| 建構子   | 不可                       | 可以             |
| 類別可擁有 | 多個介面                     | 單一父類別          |
| 共用實作  | 有限的 <code>default</code> | 完整支援           |

不要背「永遠優先介面」。共享不變量與建構流程時，抽象類別可能更直接。

PART 04

# 介面如何演進

Java 8、9、17 的關鍵能力

# default 與 static Java 8+

```
1 interface Notifier {  
2     void send(String message);  
3  
4     default void sendUrgent(String message) {  
5         send("[URGENT] " + message);  
6     }  
7  
8     static Notifier silent() {  
9         return message -> { };  
10    }  
11 }
```

## default

可覆寫的預設實作；協助 API 演進

## static

屬於介面本身；不參與動態派送

# private interface method Java 9+

```
1  interface Notifier {
2      default void sendUrgent(String message) {
3          send("[URGENT] " + normalize(message));
4      }
5
6      default void sendNormal(String message) {
7          send(normalize(message));
8      }
9
10     private String normalize(String message) {
11         return message.strip();
12     }
13
14     void send(String message);
15 }
```

`private` 方法只供介面內部重用；實作類別不可見。

# Functional Interface + lambda Java8+

```
1  @FunctionalInterface
2  interface Notifier {
3      void send(String message);
4  }
5
6  Notifier console =
7      message -> System.out.println(message);
```

## 一個抽象方法

default / static / private 不計入

## @FunctionalInterface

讓編譯器保護設計意圖

# Sealed interface Java 17 正式版

```
1 sealed interface DeliveryResult
2     permits Delivered, Rejected, Retrying {}
3
4 record Delivered(String id) implements DeliveryResult {}
5 record Rejected(String reason) implements DeliveryResult {}
6 record Retrying(int attempt) implements DeliveryResult {}
```

## 適合

封閉的領域變體集合

## 不適合

第三方自由擴充的 plugin API

Java 15、16 預覽；JEP 409 於 Java 17 定版。

# Monaco 練習：補上契約

直接編輯程式碼；此投影片不會編譯或執行 Java。

```
interface Notifier {  
    // TODO: 宣告 send(String message)  
}  
  
final class PushNotifier implements Notifier {  
    // TODO: 實作方法，印出 "PUSH > " + message  
}  
  
final class NotificationService {  
    // TODO: 以建構子注入 Notifier  
}
```

# 設計檢查清單

01 呼叫端真正需要哪個行為？

02 抽象使用領域語言，還是供應商名稱？

03 具體選擇是否集中在組裝端？

04 實作者是否被迫提供不適用的方法？

05 測試能否換入簡單 fake？

# 版本速查

| 能力                                                 | 版本      | 用途        |
|----------------------------------------------------|---------|-----------|
| <code>default</code> / <code>static</code> methods | Java 8  | 預設行為、介面工廠 |
| Functional interface / lambda                      | Java 8  | 單一行為的精簡實作 |
| <code>private</code> interface methods             | Java 9  | 介面內部重用    |
| Sealed interfaces                                  | Java 17 | 限制直接實作集合  |

規格來源：JLS §9 · JLS §9.8 · JEP 409

# 今日重點

## 契約

描述呼叫端需要的能力

## 派送

同一呼叫，依真實物件執行

## 依賴

選擇移到組裝端；箭頭指向抽象

## 測試

換入 fake，隔離外部服務

Interface 不會消除變更；它讓變更邊界更清楚。

# Lab complete.

下一步：用 `FailingNotifier` 測試失敗策略

```
$ java NotificationLab --reflect  
contract → dispatch → injection → test
```